

POLITECNICO DI BARI

DEPARTMENT OF ELECTRICAL AND INFORMATION ENGINEERING
Master Degree in Automation Engineering

Dissertation in Embedded Control

"Levitating Ball"

Supervisors
Prof. Luca De Cicco
Nunzio Barone

Candidates
Gianluca Blè
Stefano Di Lena
Giuseppe Manzo

Academic Year 2025 - 2026

Contents

1 Introduction **1**
1.1 Project Overview 1

2 Hardware Architecture and Interfacing **2**
2.1 System Overview 2
2.2 CLock Tree and Pinout Cofiguration 2
2.3 Dynamic Setpoint Iterface via ADC and DMA 4

3 Real-Time Control Loop Time-Base **5**
3.1 Theoretical Design and Parameter Sizing 5

4 Actuator Driving and PWM Frequency Optimization **7**
4.1 PWM Generation Principles 7
4.2 Initial Design: 20 kHz 7
4.3 Experimental Characterization of MOSFET Non-Linearity and Frequency Tuning 8
4.4 Final Validated PWM Configuration: 200 Hz 9

5 Position Sensing via Time-of-Flight Sensor **10**
5.1 Timing Tuning 10
5.2 Physical Characteristics: Optical Blind Zone and Static Offset 10
5.2.1 Hardware Verification via SWD 11

6 Discrete-Time PID Control Design and Software Clamping **12**
6.1 Actuator Saturation and Conditional Anti-Windup Clamping 13
6.2 Modular C Implementation 14
6.3 Interrupt-Driven Control Architecture 14

7 Conclusions **15**

Chapter 1

Introduction

1.1 Project Overview

The objective of this project is the design, implementation, and experimental validation of a control system capable of levitating and stabilizing a lightweight sphere at a desired vertical position inside a transparent cylindrical tube.

The physical plant is an inherently nonlinear and unstable aerodynamic system. An upward airflow is generated at the base of the tube by a 12 V DC fan. By modulating the fan rotational speed, the aerodynamic drag force acting on the sphere counteracts gravity. Because the tube diameter is close to the sphere diameter, the fluid-structure interaction is subject to wall boundary effects, rotational airflow vortices, and static-to-dynamic friction transitions.

From an embedded systems perspective, the control architecture follows the classic digital control loop paradigm executed at a fixed sampling period T_s:

- **Measure system output:** Acquire the vertical position of the sphere through a Time-of-Flight (ToF) laser ranging sensor.
- **Compute control law:** Evaluate a discrete-time Proportional-Integral-Derivative (PID) control algorithm based on the position error.
- **Update actuator:** Translate the computed control effort into an average voltage applied to the fan motor via Pulse Width Modulation (PWM) driving a low-side power MOSFET.

Chapter 2

Hardware Architecture and Interfacing

2.1 System Overview

The core processing unit is an STMicroelectronics NUCLEO-F446RE development board, driven by an ARM Cortex-M4 32-bit RISC microcontroller operating at a core clock frequency up to 180 MHz, equipped with an embedded single-precision Hardware Floating-Point Unit (FPU), 512 KB of Flash memory, and 128 KB of SRAM.

Actuation is provided by a high-speed DC turbine fan (TUNGFULL SHLF1209GHE-07) requiring 12 V DC and drawing up to 2.55 A under full load. The MCU cannot directly drive such current; interfacing is achieved using an LR7843 N-channel MOSFET module.

The module incorporates an onboard optocoupler isolating the low-voltage logic (3.3 V domain) from the inductive high-current supply (12 V power domain). The gate-driving PWM signal is generated on pin `PA5` via TIM2 Channel 1.

Measurement of the ping-pong ball height within the cylindrical tube is performed via a VL53L1X Time-of-Flight (ToF) laser-ranging sensor. Operating on an invisible VCSEL (Vertical-Cavity Surface-Emitting Laser) emission, the device directly computes distance based on photon travel time rather than reflected light intensity.

2.2 CLock Tree and Pinout Cofiguration

The STM32F446RE clock tree is configured using the internal 16 MHz High-Speed Internal (HSI) RC oscillator routed through the main Phase-Locked Loop (PLL). Specifically, the PLL dividers and multipliers are set to $M = 16$, $N = 336$, and $P = 4$, yielding a core system clock frequency of:

$$f_{\text{SYSCLK}} = 16 \text{ MHz} \times \frac{336}{16 \times 4} = 84 \text{ MHz}$$

Both the AHB bus (HCLK) and the APB1 timer bus (APB1 Timer Clocks) run at 84 MHz. Maintaining an exact reference clock is critical, as all hardware timers responsible for the control loop scheduling (TIM6) and PWM signal generation (TIM2) derive their counting pace from the APB1 timer bus.

Regarding the pinout configuration in STM32CubeMX, unused default pins (such as the user button on PC13, the low-speed external oscillator pins PC14/PC15, and virtual COM port pins) were reset to keep the hardware abstraction layer clean. High-speed clock

bypass pins (PH0-OSC_IN, PH1-OSC_OUT) and Serial Wire Debug / Trace pins (PA13, PA14, PB3) were preserved to ensure stable clock initialization and real-time debugging capabilities. Pin PA5 (internally connected to the board user LED LD2) was assigned to the alternate function TIM2_CH1 to output the hardware PWM signal directly to the MOSFET gate.

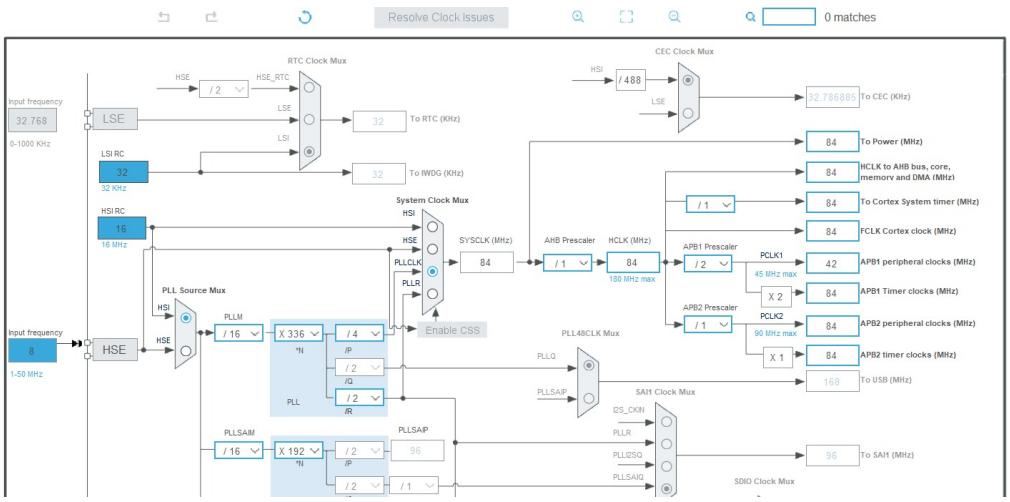


Figure 2.1: Clock Configuration CubeMX

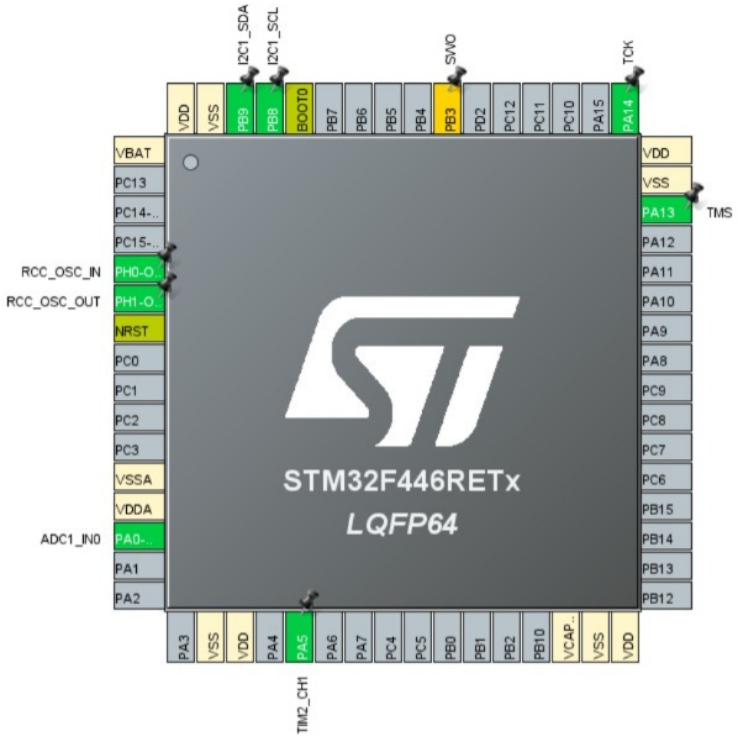


Figure 2.2: Pinout Configuration STMCubeMX

2.3 Dynamic Setpoint Iterface via ADC and DMA

To allow real-time manual control of the levitation altitude, a sliding potentiometer is interfaced through the microcontroller's internal Analog-to-Digital Converter (ADC1). To eliminate CPU overhead and ensure synchronous sampling without blocking execution, conversion is triggered continuously and readings are autonomously transferred to SRAM via the Direct Memory Access (DMA) controller operating in circular mode. A moving average filter (16 samples) is applied to the raw 12-bit digital stream (0–4095) to suppress electrical noise. The resulting filtered value is linearly mapped to the physical vertical working range of the cylindrical tube, providing a smooth, real-time reference setpoint $r[k]$ for the closed-loop control system.

Chapter 3

Real-Time Control Loop Time-Base

3.1 Theoretical Design and Parameter Sizing

Digital control algorithms are implemented as discrete-time difference equations that require execution at a strictly deterministic sampling period T_s. Relying on software delays (such as `HAL_Delay()`) blocks the CPU and introduces timing jitter due to variable code execution times. Therefore, a dedicated hardware timer must be configured as a time-base generator to trigger periodic interrupts.

For this application, Basic Timer TIM6 was selected. Unlike general-purpose or advanced timers, basic timers (TIM6 and TIM7) have no external input/output channels; they are specifically designed as 16-bit free-running up-counters intended for internal time-base generation and DAC triggering.

The target sampling frequency for the control loop is $f_s = 50$ Hz, which corresponds to a sampling period of:

$$T_s = \frac{1}{f_s} = \frac{1}{50 \text{ Hz}} = 0.02 \text{ s} = 20 \text{ ms}$$

A hardware timer connected to an internal clock bus of frequency f_{CLK} divides the input frequency through a 16-bit programmable Prescaler (PSC), producing an effective counter clock frequency:

$$f_{\text{CLK_CNT}} = \frac{f_{\text{CLK}}}{\text{PSC} + 1}$$

The 16-bit counter register (CNT) increments at each period of $f_{\text{CLK_CNT}}$ until it reaches the threshold stored in the Auto-Reload Register (ARR), at which point it resets to zero and generates an Update Event (UEV) interrupt. Consequently, the overall counting period (CP) is governed by the fundamental timer equation:

$$\text{CP} = \frac{(\text{PSC} + 1) \times (\text{ARR} + 1)}{f_{\text{CLK}}}$$

Given $f_{\text{CLK}} = 84$ MHz $= 84,000,000$ Hz on the APB1 timer bus, we must select integer values for PSC and ARR (both bounded by $2^{16} - 1 = 65535$ on a 16-bit timer) to obtain CP $= 0.02$ s.

By setting the prescaler to PSC $= 8399$, the counter clock frequency is scaled down to a convenient decimal base:

$$f_{\text{CLK_CNT}} = \frac{84,000,000 \text{ Hz}}{8399 + 1} = \frac{84,000,000 \text{ Hz}}{8400} = 10,000 \text{ Hz}$$

This implies that the timer counter increments by one step exactly every 0.1 ms (1/10, 000 Hz). To reach the desired 20 ms period, the counter must perform 200 steps. Therefore, the Auto-Reload Register is set to ARR = 199:

$$\text{CP} = \frac{(8399+1)\times(199+1)}{84,000,000\ \text{Hz}} = \frac{8400\times 200}{84,000,000\ \text{Hz}} = 0.02\ \text{s} = 20\ \text{ms}$$

Design Trade-off Remark: As established in embedded control theory, "a large value of prescaler reduces the timer's resolution; however, large prescaler values improve energy efficiency" because the downstream 16-bit counter register toggles at a much lower frequency (10 kHz instead of 84 MHz). Since TIM6 only serves as a fixed periodic interrupt generator and does not require fine duty-cycle modulation, choosing a large prescaler (PSC = 8399) and a small auto-reload value (ARR = 199) is the optimal design choice.

In STM32CubeMX, TIM6 was configured under the Timers category by enabling the Activated checkbox. In the Parameter Settings tab, the Prescaler was set to 8399, the Counter Mode was kept as Up, and the Counter Period (ARR) was set to 199. Subsequently, in the NVIC Settings tab, the TIM6 global interrupt and DAC1 and DAC2 underrun error interrupts line was enabled. Upon code generation, CubeMX automatically instantiated the TIM_HandleTypeDef structure and the initialization routine inside the C main. (Note: During the initial open-loop actuator characterization described in Section 4, the TIM6 interrupt start call was temporarily kept disabled in firmware so as to isolate and test the hardware PWM generation without interrupt interference).

Chapter 4

Actuator Driving and PWM Frequency Optimization

4.1 PWM Generation Principles

To regulate the rotational speed of the 12 V DC fan, the digital control output must be converted into an analog average voltage $V_{\text{fan}}(t_k)$. Rather than using an expensive DAC and linear power amplifier, a Pulse Width Modulated (PWM) square wave oscillating between 0 V and $V_{DD} = 3.3$ V is generated by a microcontroller timer channel to switch a low-side power MOSFET connected to the 12.16 V DC supply.

We selected General-Purpose Timer TIM2 (Channel 1) routed to pin PA5. A major advantage of using hardware PWM via TIM2 is complete CPU offloading: no interrupts are required to toggle the pin. Once initialized, the timer's Output Compare hardware peripheral autonomously generates the continuous square wave in the background. When the 50 Hz control loop will be active, the CPU will merely update the duty cycle register once every 20 ms, while the timer hardware seamlessly executes hundreds of commutation cycles without consuming any processor time.

Furthermore, TIM2 features a 32-bit Auto-Reload Register (ARR) (unlike standard 16-bit timers such as TIM3 or TIM4). In PWM generation, the core design rule is the exact opposite of the time-base timer: "Keep PSC as low as possible to allow a large ARR". Because the duty cycle is selected by writing a discrete integer value (Pulse) into the Capture/Compare Register (CCR1) within the range $[0, \text{ARR}]$, maximizing ARR maximizes the number of discrete quantization steps, ensuring ultra-fine resolution on the actuator effort. In PWM Mode 1 with up-counting configuration and High channel polarity, the output pin PA5 is driven high (3.3 V) as long as CNT $<$ CCR1, and driven low (0 V) when CCR1 $\leq$ CNT $\leq$ ARR. The resulting duty cycle (DC) and ideal average motor voltage ($\bar{V}_{\text{fan}}$) are given by:

$$DC = \frac{\text{CCR1}}{\text{ARR} + 1} \times 100\%, \qquad \bar{V}_{\text{fan}} = V_{\text{supply}} \cdot \frac{\text{CCR1}}{\text{ARR} + 1}$$

4.2 Initial Design: 20 kHz

Initially, the PWM frequency was designed at $f_{\text{PWM}} = 20$ kHz. From a theoretical standpoint, choosing a switching frequency at the upper limit of the human audible spectrum (20 kHz) prevents annoying acoustic whining from the motor windings and ensures that the inductive nature of the motor filters out high-frequency harmonics,

resulting in a smooth continuous current. Moreover, at 20 kHz, the timer completes exactly 400 full PWM periods within every 20 ms control loop step. Starting from $f_{\text{CLK}} = 84$ MHz and applying the minimum prescaler PSC = 0, the required ARR value for 20 kHz was computed as:

$$f_{\text{PWM}} = \frac{f_{\text{CLK}}}{(\text{PSC}+1) \times (\text{ARR}+1)} = \frac{84,000,000}{(0+1) \times (4199+1)} = \frac{84,000,000}{4200} = 20,000 \text{ Hz}$$

In STM32CubeMX, PA5 was set to TIM2_CH1, Channel 1 was set to PWM Generation CH1, PSC = 0, ARR = 4199, Pulse (CCR1) = 0 (ensuring the fan starts completely off at boot), PWM Mode 1, and CH Polarity = High. To test the actuator in open loop and measure the terminal voltage with a digital multimeter, two HAL instructions were placed inside main.c prior to the infinite loop:

```
// Set duty cycle (50% with ARR = 4199 -> CCR1 = 2100)
__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 2100);

// Start PWM generation on TIM2 Channel 1 (PA5)
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
```

4.3 Experimental Characterization of MOSFET Non-Linearity and Frequency Tuning

During experimental testing with the 12.16 V power supply and the low-cost MOSFET driver module, we observed severe non-linear behavior at 20 kHz. Due to the internal circuit topology of the budget MOSFET module (specifically slow gate discharge times and turn-off propagation delays, $t_{\text{off}} \gg t_{\text{on}}$), a 50 μs switching period (20 kHz) was too fast for the transistor to switch off cleanly.We therefore conducted a systematic experimental campaign, progressively lowering the PWM frequency and comparing the expected average voltage against the actual voltage measured across the fan terminals with a multimeter:

- Test at 1 kHz (PSC = 0, ARR = 83999): With a supply voltage of 12.16 V at 100% duty cycle, we tested the low-thrust region required to levitate the sphere:
  - At 5% Duty Cycle (CCR1 = 4200), the fan remained completely stalled due to static friction and motor dead-zone; the sphere stayed at the bottom of the tube.
  - At 10% Duty Cycle (CCR1 = 8400), the theoretical expected voltage was $\bar{V}_{\text{expected}} = 0.10 \times 12.16$ V = 1.22 V. However, the multimeter measured 2.37 V (nearly double the commanded value, equivalent to $\sim$ 19.5% real duty cycle). At this voltage, the sphere violently shot to the top of the tube.

  Physical and Aerodynamic Issue: Because the sphere had to be controlled within the narrow 5%–10% window, and because the narrow tube diameter caused the sphere to spin against the inner walls under the fan's helical airflow, any slight activation above the 5% stall threshold immediately resulted in excessive voltage due to MOSFET distortion, making smooth levitation impossible.

- Test at 840 Hz (PSC = 0, ARR = 99999): Setting a 50% duty cycle (CCR1 = 50,000), the expected voltage was $\sim$ 6.08 V, whereas the measured terminal voltage was 7.0 V, still showing an unacceptable error of nearly +1 V.

- Test at 420 Hz (PSC = 1, ARR = 99999): At 50% duty cycle (CCR1 = 50,000), the measured voltage dropped to 6.6 V compared to the expected 6.08 V, confirming that lengthening the PWM period reduces the relative weight of the MOSFET turn-off delay.
- Test at 280 Hz (PSC = 2, ARR = 99999): At 50% duty cycle (CCR1 = 50,000), the measured voltage reached 6.4 V (an error of $\sim$ 0.3 V). At 75% duty cycle (CCR1 = 75,000), we measured 9.48 V against an expected 9.12 V.

4.4 Final Validated PWM Configuration: 200 Hz

Based on the experimental trend, 200 Hz (switching period $T_{\text{PWM}} = 5$ ms $= 5000$ μs) was identified as the optimal trade-off: it is slow enough to render the MOSFET switching delays negligible, yet fast enough (4× faster than the 50 Hz control loop) to maintain smooth mechanical inertia in the brushless fan. Exploiting the 32-bit counter capability of TIM2, we kept the prescaler at its minimum (PSC = 0) and set ARR = 419999:

$$f_{\text{PWM}} = \frac{84,000,000 \text{ Hz}}{(0+1) \times (419,999+1)} = \frac{84,000,000 \text{ Hz}}{420,000} = \mathbf{200.00} \text{ Hz}$$

This configuration yields a 0% mathematical frequency error and provides 420,000 discrete resolution steps for duty cycle control. The exact CCR1 values used for our open-loop validation tests are derived directly from the 420,000 base count:

- 5% Duty Cycle: CCR1 $= 0.05 \times 420,000 = \mathbf{21,000}$
- 10% Duty Cycle: CCR1 $= 0.10 \times 420,000 = \mathbf{42,000}$
- 50% Duty Cycle: CCR1 $= 0.50 \times 420,000 = \mathbf{210,000}$
- 75% Duty Cycle: CCR1 $= 0.75 \times 420,000 = \mathbf{315,000}$

Experimental verification at 200 Hz confirmed a drastic improvement in linearity: across the tested operating points, the discrepancy between the commanded duty cycle and the effective duty cycle calculated from multimeter voltage readings was reduced to approximately 0.1%, with a maximum absolute voltage deviation of at most 0.2 V.

Consequently, 200 Hz (PSC = 0, ARR = 419999) was locked in as the final actuator configuration. In addition to lowering the PWM frequency to 200 Hz, the initial sphere was replaced with one featuring a more suitable mass-to-diameter ratio relative to the tube section. Combined with the restored MOSFET linearity at 200 Hz, this mechanical adjustment eliminated wall-spinning instabilities and provided a smooth, continuous, and fully controllable aerodynamic lift response across the duty cycle range.

Chapter 5

Position Sensing via Time-of-Flight Sensor

Altitude tracking of the sphere is performed by an STMicroelectronics VL53L1X Time-of-Flight (ToF) sensor. The sensor interfaces with the STM32F446RE via the I2C1 peripheral routed to pins PB8 (SCL) and PB9 (SDA). Following the lecture notes on I2C communication, the 7-bit slave address 0x29 is converted to an 8-bit HAL address via a 1-bit left shift:

$$\text{ADDR}_{\text{HAL}} = 0\text{x}29 \ll 1 = 0\text{x}52$$

In STM32CubeMX, I2C1 is configured in Fast Mode (400 kHz). At 400 kHz, reading the 2-byte distance register requires under 0.15 ms, introducing negligible processing overhead.

5.1 Timing Tuning

During early standalone testing using software delays, polling the sensor produced stable readings. However, once the periodic hardware interrupt of TIM6 was activated at 40 Hz (25 ms), the measurement process suffered from severe data corruption. Because the sensor requires a minimum of 20 ms for internal photon integration, a 25 ms interrupt cycle left insufficient margin for internal processing and I2C register updates.By decreasing the execution frequency to ≈ 37 Hz ($T_s \approx 26.57$ ms). This change resolved the timing collisions and restored deterministic acquisition across all samples.

5.2 Physical Characteristics: Optical Blind Zone and Static Offset

Bench characterization both in open air and within the vertical tube identified two physical optical constraints:

1. Optical Parallax Blind Zone (< 30 mm): Because the emitter and receiver are spaced several millimeters apart on the PCB, optical parallax prevents photons reflected closer than 30 mm from hitting the optical receiver, causing the sensor to return 0 mm.

2. Systematic Offset ($\approx$ 10–15 mm): Inside the cylinder, peripheral photon reflections against the transparent walls introduce a constant static positive offset.

   Because the discrete-time PID feedback operates purely on the relative tracking error $e[k] = r[k] - y[k]$, a minor fixed offset does not degrade closed-loop regulation or stability.

5.2.1 Hardware Verification via SWD

Sensor functionality was validated over SWD using STM32CubeIDE Live Expressions without blocking serial print routines:

- sensor_id: Evaluated as 0xEACC, validating correct physical bus connections and 16-bit register addressing.
- sensor_ok: Asserted to 1, confirming successful transmission of the 91-byte initialization sequence.
- distance_mm: Dynamically validated in real-time, providing consistent millimeter readings with an acceptable static offset of $\approx$ 10–15 mm across the operational tube height.

Chapter 6

Discrete-Time PID Control Design and Software Clamping

To stabilize the sphere at a desired vertical reference position $r(t)$ despite gravity and aerodynamic disturbances, a Proportional-Integral-Derivative (PID) feedback controller is employed.

Let $y(t)$ denote the measured vertical position of the sphere and $e(t) = r(t) - y(t)$ denote the tracking error. In the continuous-time domain, the ideal parallel PID control law is expressed as:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d \frac{de(t)}{dt}$$

where K_p, K_i, and K_d represent the proportional, integral, and derivative gains, respectively. Each term fulfills a specific physical role in the levitation plant:

- **Proportional action (P):** Provides an immediate corrective thrust proportional to the current position error. However, because the sphere has a non-zero weight mg, a purely proportional controller would always require a non-zero steady-state error to sustain the hovering fan speed
- **Integral action (I):** Accumulates past tracking errors over time to automatically estimate and maintain the exact baseline PWM duty cycle required to counteract gravity at equilibrium, driving the steady-state position error to zero ($e_\infty = 0$).
- **Derivative action (D):** Acts as a predictive aerodynamic damper by reacting to the rate of change of the error. When the sphere approaches the setpoint rapidly, the derivative term becomes negative and reduces fan thrust early, preventing overshoot and vertical oscillations inside the tube.

Because the control algorithm is executed digitally inside the microcontroller at discrete time instants $t_k = kT_s$ (with sampling period $T_s = 0.02$ s enforced by TIM6), the continuous-time operators must be discretized into difference equations:

1. Tracking Error at step k:
$$e[k] = r[k] - y[k]$$
2. Proportional Term:
$$P[k] = K_p \cdot e[k]$$

3. Integral Term (Rectangular / Forward Euler approximation): The continuous integral is approximated by accumulating the area of discrete rectangles of width T_s and height $e[k]$:
$$I[k] = I[k-1] + K_i \cdot T_s \cdot e[k]$$

4. Derivative Term (Backward Finite Difference approximation): The time derivative of the error is approximated using the first-order backward difference between the current error $e[k]$ and the previous sample error $e[k-1]$:
$$D[k] = K_d \cdot \frac{e[k] - e[k-1]}{T_s}$$

Combining the three contributions yields the unconstrained discrete-time control output at time step k:
$$u_{\text{tentative}}[k] = P[k] + I[k] + D[k]$$

6.1 Actuator Saturation and Conditional Anti-Windup Clamping

In any physical control system, the actuator has finite operating boundaries. Specifically, the PWM duty cycle driving the fan cannot exceed 100% (or a safer upper threshold out_max to prevent shooting the sphere out of the tube) and cannot drop below 0% (or a minimum threshold out_min corresponding to the fan stall region)

When a large setpoint step change occurs or if the sphere is temporarily held down at the bottom of the tube, the computed control output $u_{\text{tentative}}[k]$ quickly exceeds out_max. If the integral state $I[k]$ were allowed to keep accumulating positive error while the actuator is already physically saturated at its maximum limit, the integrator memory would grow to an abnormally large value—a phenomenon known as Integral Windup. Once the sphere finally reaches and crosses the setpoint, a wound-up integrator would take several seconds to discharge, keeping the fan at maximum thrust and causing severe overshoot and instability. To prevent this issue, our firmware implements a Conditional Integration (Clamping Anti-Windup) scheme. At each sampling step k, a tentative integral value is first computed:
$$I_{\text{tentative}}[k] = I[k-1] + K_i \cdot T_s \cdot e[k]$$

Before committing $I_{\text{tentative}}[k]$ to the permanent controller memory, the total output $u[k] = P[k] + I_{\text{tentative}}[k] + D[k]$ is checked against the saturation bounds:

- Upper Saturation ($u[k] > u_{\max}$): The actual output sent to the actuator is clamped to $u[k] = u_{\max}$. The integral state is updated ($I[k] = I_{\text{tentative}}[k]$) if and only if the current error is negative ($e[k] < 0$), meaning the error is helping to drive the controller out of upper saturation. Otherwise, if $e[k] \geq 0$, integration is frozen ($I[k] = I[k-1]$).

- Lower Saturation ($u[k] < u_{\min}$): The actual output is clamped to $u[k] = u_{\min}$. The integral state is updated if and only if the error is positive ($e[k] > 0$), which pulls the controller out of lower saturation; otherwise, integration is frozen.

- Linear Region ($u_{\min} \leq u[k] \leq u_{\max}$): The actuator is not saturated, so the integral state is updated normally ($I[k] = I_{\text{tentative}}[k]$).

6.2 Modular C Implementation

To keep the firmware modular, reusable, and free from scattered global variables, the PID controller was encapsulated into a dedicated C module (pid.h and pid.c). A custom data structure PID_Controller stores the tuning gains (Kp, Ki, Kd), the sampling period (Ts), the saturation limits (out_min, out_max), and the internal state memory across sampling intervals (integrator for I[k-1] and prev_error for e[k-1]):

```
typedef struct {
    float Kp;
    float Ki;
    float Kd;
    float Ts;             // Sampling time in seconds
    float out_min;        // Lower saturation limit
    float out_max;        // Upper saturation limit
    float integrator;     // Integral state memory I[k-1]
    float prev_error;     // Previous step error memory e[k-1]
} PID_Controller;
```

Three functions operate on this structure via pointer passing (PID_Controller *pid).

- PID_Init(): Initializes the controller parameters and clears the internal state variables at system startup.
- PID_Reset(): Resets integrator and prev_error to 0.0f whenever the control loop is restarted.
- PID_Update(PID_Controller *pid, float setpoint, float measurement): Executed periodically every 26, 6 ms, it computes the tracking error, evaluates the discrete P, I, and D terms, applies the conditional anti-windup clamping logic, updates prev_error = error for the next cycle, and returns the clamped control action.

6.3 Interrupt-Driven Control Architecture

PID_Update(PID_Controller *pid, float setpoint, float measurement): Executed periodically every 26, 6 ms, it computes the tracking error, evaluates the discrete P, I, and D terms, applies the conditional anti-windup clamping logic, updates prev_error = error for the next cycle, and returns the clamped control action.

1. Every $T_s = 26, 6$ ms, TIM6 overflows and triggers

   HAL_TIM_PeriodElapsedCallback() interrupt service routine, which sets a volatile synchronization flag.

2. Inside the while(1) background loop in main.c, the CPU checks if (loop_flag == 1), immediately clears the flag, acquires the current sphere position y[k] and reference setpoint r[k], invokes PID_Update(), and writes the resulting control effort into the TIM2 capture/compare register via

   ```
   __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, ccr).
   ```

Chapter 7

Conclusions

The closed-loop levitation system has been fully implemented, integrated, and experimentally validated.

- **Closed-Loop Performance:** By operating the discrete-time control loop at a deterministic sampling rate of 37.5 Hz ($T_s \approx 26.6$ ms) via TIM6, combined with the optimized 200 Hz hardware PWM actuator signal on TIM2 and the dynamic analog setpoint provided by the potentiometer-ADC-DMA chain, the system achieves smooth and stable sphere levitation.
- **Disturbance Rejection:** The integration of the anti-windup clamping algorithm effectively prevents integral saturation during large setpoint steps, eliminating severe overshoots.

The combination of precise optical altitude tracking via the VL53L1X sensor, proper actuator linearization, and correctly tuned PID gains (K_p, K_i, K_d) ensures rapid transient response, precise steady-state tracking, and robust rejection of aerodynamic disturbances inside the vertical tube. The project successfully demonstrates the practical application of embedded real-time control theory.

Bibliography

[1] L. D. Cicco, "Embedded control: Course slides." Lecture notes, 2024–2025.

[2] STMicroelectronics, *RM0390 Reference Manual: STM32F446xx advanced Arm®-based 32-bit MCUs*. STMicroelectronics, rev. 6 ed., 2021. DocID RM0390.

[3] STMicroelectronics, *DS10693 Datasheet: STM32F446xC/E Arm® Cortex®-M4 32-bit MCU+FPU*. STMicroelectronics, rev. 10 ed., 2021. DocID DS10693.

[4] STMicroelectronics, *VL53L1X: Time-of-Flight ranging sensor with advanced miniature optics*. STMicroelectronics, rev. 7 ed., 2021. DocID DS12285.